Hammered Lab

Q1:

¿Qué servicio utilizaron los atacantes para acceder al sistema?

→ Revise los registros (Logs) del sistema para detectar errores de autenticación. Estos registros pueden ayudarle a identificar el servicio que los atacantes intentaron usar para acceder.

→ El archivo `auth.log` contiene registros de los intentos de autenticación. Busque fallos de autenticación repetidos para un servicio específico.

→ Utilice `grep "fail" auth.log` para identificar los intentos de autenticación fallidos.

```
Apr 26 06:11:30 app-1 sshd[21221]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=219.139.243.236  user=root
Apr 26 06:11:34 app-1 sshd[21223]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=219.139.243.236  user=root
Apr 26 06:11:39 app-1 sshd[21225]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=219.139.243.236  user=root
Apr 26 06:11:43 app-1 sshd[21227]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=219.139.243.236  user=root
Apr 26 06:11:47 app-1 sshd[21229]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=219.139.243.236  user=root
Apr 26 06:11:52 app-1 sshd[21231]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=219.139.243.236  user=root
Apr 26 06:11:58 app-1 sshd[21233]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=219.139.243.236  user=root
Apr 26 06:12:04 app-1 sshd[21235]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=219.139.243.236  user=root
Apr 26 06:12:08 app-1 sshd[21245]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=219.139.243.236  user=root
Apr 26 06:12:16 app-1 sshd[21247]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=219.139.243.236  user=root
Apr 26 06:12:22 app-1 sshd[21249]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=219.139.243.236  user=root
Apr 26 06:12:31 app-1 sshd[21251]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=219.139.243.236  user=root
Apr 26 07:18:07 app-1 sshd[22034]: reverse mapping checking getaddrinfo for abts-tn-static-200.9.165.122.airtelbroadband.in [122.165.9.200] failed - POSSIBLE BREAK-IN A
TTEMPT!
Apr 26 07:18:07 app-1 sshd[22034]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=122.165.9.200  user=root
Apr 26 07:18:11 app-1 sshd[22036]: reverse mapping checking getaddrinfo for abts-tn-static-200.9.165.122.airtelbroadband.in [122.165.9.200] failed - POSSIBLE BREAK-IN A
TTEMPT!
Apr 26 07:18:11 app-1 sshd[22036]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=122.165.9.200
Apr 26 07:18:16 app-1 sshd[22038]: reverse mapping checking getaddrinfo for abts-tn-static-200.9.165.122.airtelbroadband.in [122.165.9.200] failed - POSSIBLE BREAK-IN A
TTEMPT!
Apr 26 07:18:16 app-1 sshd[22038]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=122.165.9.200
Apr 26 07:18:24 app-1 sshd[22040]: reverse mapping checking getaddrinfo for abts-tn-static-200.9.165.122.airtelbroadband.in [122.165.9.200] failed - POSSIBLE BREAK-IN A
TTEMPT!
Apr 26 07:18:24 app-1 sshd[22040]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=122.165.9.200
Apr 26 07:18:31 app-1 sshd[22042]: reverse mapping checking getaddrinfo for abts-tn-static-200.9.165.122.airtelbroadband.in [122.165.9.200] failed - POSSIBLE BREAK-IN A
TTEMPT!
Apr 26 07:18:31 app-1 sshd[22042]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=122.165.9.200
Apr 26 07:18:35 app-1 sshd[22044]: reverse mapping checking getaddrinfo for abts-tn-static-200.9.165.122.airtelbroadband.in [122.165.9.200] failed - POSSIBLE BREAK-IN A
TTEMPT!
Apr 26 07:18:35 app-1 sshd[22044]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=122.165.9.200  user=mysql
Apr 26 07:18:44 app-1 sshd[22046]: reverse mapping checking getaddrinfo for abts-tn-static-200.9.165.122.airtelbroadband.in [122.165.9.200] failed - POSSIBLE BREAK-IN A
TTEMPT!
Apr 26 07:18:44 app-1 sshd[22046]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=122.165.9.200
Apr 26 08:23:45 app-1 sshd[22662]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=65.208.122.48
Apr 26 08:23:49 app-1 sshd[22664]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=65.208.122.48
```

La salida muestra como los atacantes estaban intentando acceder al sistema mediante el servicio **`SSH`**, evidenciado por múltiples intentos fallidos de autenticación registrados en **`auth.log`**.

Respuesta de Q1: SSH

Q2:

¿Cuál es la versión del sistema operativo del sistema de destino?

→ El registro `dmesg` registra los mensajes del kernel y del sistema, incluidos detalles sobre la versión del sistema operativo durante el arranque.

→ Ejecute `head dmesg` para obtener información sobre la versión de Linux. Busque la línea que contiene la **versión de Linux**, que revelará la versión del sistema operativo.

```
checho@Windows-localhost:~/Hammered$ head dmesg
[    0.000000] Initializing cgroup subsys cpuset
[    0.000000] Initializing cgroup subsys cpu
[    0.000000] Linux version 2.6.24-26-server (buildd@crested) (gcc version 4.2.4 (Ubuntu 4.2.4-1ubuntu3)) #1 SMP Tue Dec 1 18:26:43 UTC 2009 (Ubuntu 2.6.24-26.64-serve
r)
[    0.000000] Command line: root=UUID=a691743a-a4b7-482d-95ff-406e5acd83a3 ro quiet splash
[    0.000000] BIOS-provided physical RAM map:
[    0.000000]  BIOS-e820: 0000000000000000 - 000000000009f800 (usable)
[    0.000000]  BIOS-e820: 000000000009f800 - 00000000000a0000 (reserved)
[    0.000000]  BIOS-e820: 00000000000ca000 - 00000000000cc000 (reserved)
[    0.000000]  BIOS-e820: 00000000000dc000 - 00000000000e4000 (reserved)
[    0.000000]  BIOS-e820: 00000000000e8000 - 0000000000100000 (reserved)
checho@Windows-localhost:~/Hammered$
```

La versión visible asociada a Ubuntu dentro del log es:

```
0.000000] Linux version 2.6.24-26-server (buildd@crested) (gcc version 4.2.4 (Ubuntu 4.2.4-1ubuntu3)) #1 SMP Tue Dec 1 18:26:43 UTC 2009 (Ubuntu
```

Respuesta de Q2: 4.2.4-1ubuntu3

Q3:

¿Cuál es el nombre de la cuenta comprometida?

→ Consulte el archivo `auth.log` para encontrar información sobre las cuentas comprometidas. Céntrese en las cuentas con intentos de inicio de sesión inusuales o frecuentes.

→ Busque intentos de autenticación relacionados con cuentas con `uid=0`, que normalmente se asocian con cuentas privilegiadas.

¿Que es una cuenta comprometida?

No es cualquier usuario en los registros, corresponde a un usuario que recibió muchos **intentos de autenticaciones fallidas** y luego obtuvo **acceso de forma exitosa**.

`uid=0` no es solo otro usuario, Es el superusuario (root) quien tiene permisos para:

- Leer o modificar archivos
- Instalar malware
- Crear usuarios
- Borrar registros (Logs)
- Tener el control total del sistema

→ Ejecute `grep "uid=0" auth.log` todas las líneas donde aparezca `uid=0` dentro del archivo `auth.log`

Primera fase: se evidencia diversos intentos de autenticación mediante usuario root desde diversas direcciones IP externas.

```
Apr 24 11:28:56 app-1 sshd[24177]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=121.11.66.70  user=root
Apr 24 11:29:00 app-1 sshd[24179]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=121.11.66.70  user=root
Apr 24 11:29:04 app-1 sshd[24181]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=121.11.66.70  user=root
Apr 24 11:29:12 app-1 sshd[24183]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=121.11.66.70  user=root
```

```
Apr 23 17:21:15 app-1 sshd[17878]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=124.207.117.9
Apr 23 17:21:19 app-1 sshd[17880]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=124.207.117.9
Apr 23 17:21:23 app-1 sshd[17882]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=124.207.117.9
Apr 23 17:21:27 app-1 sshd[17884]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=124.207.117.9
Apr 23 17:21:30 app-1 sshd[17886]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=124.207.117.9
```

```
Apr 24 03:33:40 app-1 sshd[21407]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=211.154.254.248  user=root
Apr 24 03:33:44 app-1 sshd[21409]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=211.154.254.248
Apr 24 03:33:48 app-1 sshd[21411]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=211.154.254.248  user=root
Apr 24 03:33:53 app-1 sshd[21413]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=211.154.254.248  user=root
Apr 24 03:33:57 app-1 sshd[21415]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=211.154.254.248  user=root
Apr 24 03:34:01 app-1 sshd[21417]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=211.154.254.248  user=root
Apr 24 03:34:06 app-1 sshd[21420]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=211.154.254.248
Apr 24 03:34:11 app-1 sshd[21422]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=211.154.254.248  user=root
Apr 24 03:34:15 app-1 sshd[21424]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=211.154.254.248  user=root
```

Segunda fase: inicios exitosos de autenticación mediante usuario root.

```
Apr 25 01:46:01 app-1 CRON[7333]: pam_unix(cron:session): session opened for user root by (uid=0)
Apr 25 01:51:01 app-1 CRON[7340]: pam_unix(cron:session): session opened for user root by (uid=0)
Apr 25 01:51:01 app-1 CRON[7343]: pam_unix(cron:session): session opened for user root by (uid=0)
Apr 25 01:56:01 app-1 CRON[7351]: pam_unix(cron:session): session opened for user root by (uid=0)
Apr 25 01:56:01 app-1 CRON[7354]: pam_unix(cron:session): session opened for user root by (uid=0)
Apr 25 02:01:01 app-1 CRON[7361]: pam_unix(cron:session): session opened for user root by (uid=0)
Apr 25 02:01:01 app-1 CRON[7364]: pam_unix(cron:session): session opened for user root by (uid=0)
```

El mensaje `session opened for user root by (uid=0)` significa que se ha iniciado una sesión con privilegios máximos de **administrador (root)**.

Respuesta de Q3: root

Q4:

¿Cuántos atacantes, representados por direcciones IP únicas, pudieron acceder con éxito al sistema después de intentos fallidos iniciales?

→ Investigue intentos de inicio de sesión exitosos desde diferentes direcciones IP en los registros de autenticación.

→ Utilice `auth.log` para contar cuántas direcciones IP únicas tuvieron inicios de sesión exitosos.

→ Primero, enumere los intentos fallidos con `grep "authentication failure" auth.log`, y luego enumere los inicios de sesión exitosos usando `grep "Accepted password" auth.log` para identificar cuántas direcciones IP distintas accedieron al sistema.

Al ejecutar `grep "Accepted password" auth.log | cut -d ' ' -f 9 | sort | uniq` nos mostrara una lista sin repetir de los usuarios que lograron autenticarse por SSH con éxito.

```
checho@Windows-localhost:~/Hammered$ grep "Accepted password" auth.log | cut -d ' ' -f 9 | sort | uniq
dhg
fido
for  ──► For no corresponde a un usuario
root
user1
user2
user3
```

Respuesta de Q4: 6

Q5:

¿Qué dirección IP del atacante inició sesión exitosamente en el sistema la mayor cantidad de veces?

→ Concéntrese en el archivo `auth.log` para determinar qué dirección IP tuvo la mayor cantidad de inicios de sesión exitosos.

→ Ordene los intentos de inicio de sesión exitosos por dirección IP para ver cuál aparece con más frecuencia.

→ Utilice `grep "Accepted password for" auth.log | awk '{print $11}' | sort | uniq -c` para contar cuántas veces inició sesión cada IP. Identifique la dirección IP con la mayor cantidad de inicios de sesión.

La salida de `grep "Accepted password for" auth.log | awk '{print $11}' | sort | uniq -c` muestra diversas direcciones IP.

```
checho@Windows-localhost:~/Hammered$ grep "Accepted password for" auth.log | awk '{print $11}' | sort | uniq -c
     14 10.0.1.2
      4 10.0.1.4
      2 121.11.66.70
      2 122.226.202.12
      1 151.81.204.141
      1 151.81.205.100
      1 151.82.3.201
      1 166.129.196.88
      1 188.131.22.69
      4 188.131.23.37
     23 190.166.87.164
      1 190.167.70.87
      3 190.167.74.184
      3 192.168.126.1
      1 193.1.186.197
      1 201.229.176.217
      2 208.80.69.69
      1 208.80.69.70
      7 208.80.69.74
      4 219.150.161.20
      1 222.169.224.197
      1 222.66.204.246
      1 61.168.227.12
      2 65.195.182.120
      6 65.88.2.5
      1 67.164.72.181
      5 71.132.129.212
     22 76.191.195.140
      2 94.52.185.9
```

Hay que tener en consideración que no todas las direcciones IP que se muestran en la lista son maliciosas, por lo que no son utilizadas por el atacante.

¿Que criterios se deben aplicar para identificar la IP utilizada por el atacante?

1) - Identificar direcciones IP externas y aislar direcciones privadas:

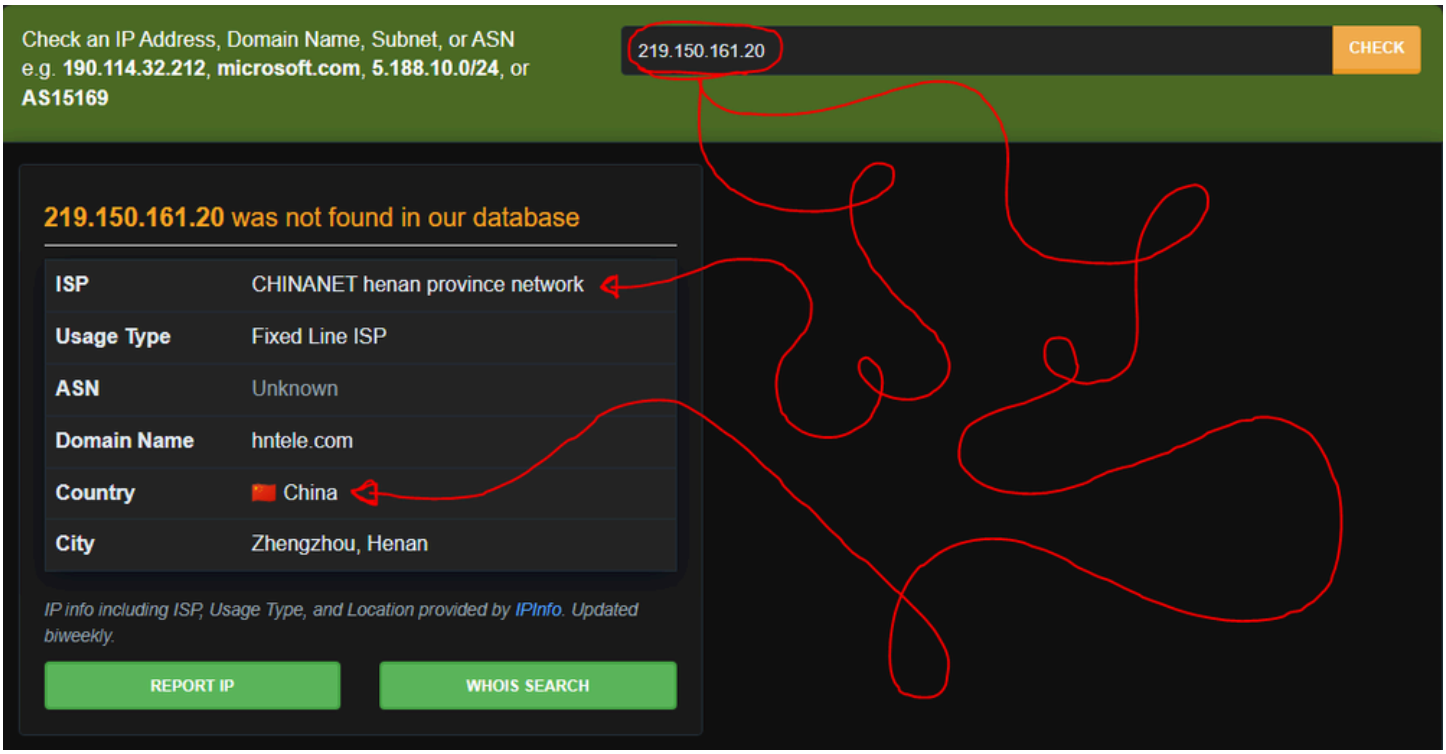
- Las direcciones IP internas que se muestran en la lista son**: 10.0.1.2 | 10.0.1.4 | 192.168.126.1**

2) - Verificación de reputación de la IP en fuentes de Threat Intelligence:

- Mediante el análisis de fuentes de Threat Intelligence, la dirección IP del atacante es la **219.150.161.20**

La dirección IP pertenece a infraestructura común:

- **ISP**: **CHINANET**
- **Ubicación geográfica**: **China**



Esto no implica que el atacante esté físicamente allí. Es probable que la IP forme parte de un sistema comprometido o una botnet, comúnmente utilizada para lanzar ataques automatizados desde diversas ubicaciones globales.

La ejecución de `grep "Accepted password" auth.log | grep "219.150.161.20"` muestra todas las veces que que la IP **219.150.161.20** logró autenticarse exitosamente en el sistema.

```
checho@Windows-localhost:~/Hammered$ grep "Accepted password" auth.log | grep "219.150.161.20"
Apr 19 05:41:44 app-1 sshd[8810]: Accepted password for root from 219.150.161.20 port 51249 ssh2
Apr 19 05:42:27 app-1 sshd[9031]: Accepted password for root from 219.150.161.20 port 40877 ssh2
Apr 19 05:55:20 app-1 sshd[12996]: Accepted password for root from 219.150.161.20 port 55545 ssh2
Apr 19 05:56:05 app-1 sshd[13218]: Accepted password for root from 219.150.161.20 port 36585 ssh2
```

Los registros muestran múltiples inicios de sesión SSH exitosos a la cuenta de **root** desde la dirección IP **219.150.161.20** en un corto período de tiempo.

Respuesta de Q5: 219.150.161.20

Q6:

¿Cuántas solicitudes se enviaron al servidor Apache?

→ Debe analizar los registros de acceso del servidor Apache para encontrar la cantidad de solicitudes.

→ Consulte el archivo `apache2/www-access.log`, que registra todas las solicitudes al servidor Apache.

→ Ejecute `cat www-access.log | wc -l` para contar el número total de solicitudes en el registro.

La salida muestra que la cantidad total de solicitudes enviadas hacia el servidor apache es de 365 solicitudes:

```
checho@Windows-localhost:~/Hammered/apache2$ cat www-access.log | wc -l
365
```

Respuesta de Q6: 365

Q7:

¿Cuántas reglas se han agregado al firewall?

→ La actividad y los cambios del firewall suelen registrarse en archivos del sistema como `auth.log`. Busque entradas de registro relacionadas con `iptables`.

→ Busque entradas relacionadas con las adiciones de reglas de firewall, específicamente `iptables`, en los archivos de registro.

→ Utilice `grep "iptables" auth.log` para encontrar todas las entradas relacionadas con las reglas del firewall y contar la cantidad de reglas agregadas.

```
checho@Windows-localhost:~/Hammered$ grep "iptables" auth.log
Apr 15 12:49:09 app-1 sudo:   user1 : TTY=pts/0 ; PWD=/opt/software/web/app ; USER=root ; COMMAND=/usr/bin/tee ../templates/proxy/iptables.conf
Apr 15 15:06:13 app-1 sudo:   user1 : TTY=pts/1 ; PWD=/opt/software/web/app ; USER=root ; COMMAND=/usr/bin/tee ../templates/proxy/iptables.conf
Apr 15 15:17:45 app-1 sudo:   user1 : TTY=pts/1 ; PWD=/opt/software/web/app ; USER=root ; COMMAND=/usr/bin/tee ../templates/proxy/iptables.conf
Apr 15 15:18:23 app-1 sudo:   user1 : TTY=pts/1 ; PWD=/opt/software/web/app ; USER=root ; COMMAND=/usr/bin/tee ../templates/proxy/iptables.conf
Apr 24 19:25:37 app-1 sudo:     root : TTY=pts/2 ; PWD=/etc ; USER=root ; COMMAND=/sbin/iptables -L
Apr 24 20:03:06 app-1 sudo:     root : TTY=pts/2 ; PWD=/etc ; USER=root ; COMMAND=/sbin/iptables -A INPUT -p ssh -dport 2424 -j ACCEPT
Apr 24 20:03:44 app-1 sudo:     root : TTY=pts/2 ; PWD=/etc ; USER=root ; COMMAND=/sbin/iptables -A INPUT -p tcp -dport 53 -j ACCEPT
Apr 24 20:04:13 app-1 sudo:     root : TTY=pts/2 ; PWD=/etc ; USER=root ; COMMAND=/sbin/iptables -A INPUT -p udp -dport 53 -j ACCEPT
Apr 24 20:06:22 app-1 sudo:     root : TTY=pts/2 ; PWD=/etc ; USER=root ; COMMAND=/sbin/iptables -A INPUT -p tcp --dport ssh -j ACCEPT
Apr 24 20:11:00 app-1 sudo:     root : TTY=pts/2 ; PWD=/etc ; USER=root ; COMMAND=/sbin/iptables -A INPUT -p tcp --dport 53 -j ACCEPT
Apr 24 20:11:08 app-1 sudo:     root : TTY=pts/2 ; PWD=/etc ; USER=root ; COMMAND=/sbin/iptables -A INPUT -p tcp --dport 113 -j ACCEPT
```

Primera observación general:

Las reglas agregadas al firewall fueron añadidas mediante el **usuario privilegiado (root)**.

```
root : TTY=pts/2 ; PWD=/etc ; USER=root ; COMMAND=/sbin/iptables -L
root : TTY=pts/2 ; PWD=/etc ; USER=root ; COMMAND=/sbin/iptables -A INPUT -p ssh -dport 2424 -j ACCEPT
root : TTY=pts/2 ; PWD=/etc ; USER=root ; COMMAND=/sbin/iptables -A INPUT -p tcp -dport 53 -j ACCEPT
root : TTY=pts/2 ; PWD=/etc ; USER=root ; COMMAND=/sbin/iptables -A INPUT -p udp -dport 53 -j ACCEPT
root : TTY=pts/2 ; PWD=/etc ; USER=root ; COMMAND=/sbin/iptables -A INPUT -p tcp --dport ssh -j ACCEPT
root : TTY=pts/2 ; PWD=/etc ; USER=root ; COMMAND=/sbin/iptables -A INPUT -p tcp --dport 53 -j ACCEPT
root : TTY=pts/2 ; PWD=/etc ; USER=root ; COMMAND=/sbin/iptables -A INPUT -p tcp --dport 113 -j ACCEPT
```

Los registros muestran que el atacante, operando con privilegios de root, modificó las reglas del firewall usando iptables. Se agregaron varias reglas INPUT, incluyendo la apertura de puertos SSH no estándar (por ejemplo, 2424) y puertos DNS (53 TCP/UDP), lo que puede indicar intentos de establecer persistencia, habilitar el acceso remoto o crear canales de comunicación encubiertos.

Se agregaron un total de **6 reglas de firewall**.

Respuesta de Q7: 6

Q8:

Uno de los archivos descargados en el sistema de destino es una herramienta de escaneo. ¿Cómo se llama?

→ Revise los registros relacionados con la instalación o descarga de paquetes en el sistema. Es posible que el atacante haya instalado una reconocimiento.

→ El archivo `apt/term.log` registra los paquetes instalados. Busque en este registro las herramientas instaladas relacionadas con el análisis.

Le ejecución de `grep -i "nmap" term.log` muestra el proceso de proceso de instalación con APT de la herramienta nmap.

```
checho@Windows-localhost:~/Hammered/apt$ grep -i "nmap" term.log
Selecting previously deselected package nmap.
Unpacking nmap (from .../archives/nmap_4.53-3_amd64.deb) ...
Setting up nmap (4.53-3) ...
```

El archivo **`apt/term.log`** muestra que el paquete **nmap** se instaló en el sistema. Nmap es una herramienta de escaneo de red muy conocida, lo que indica que el atacante realizó actividades de reconocimiento después de comprometer el sistema.

Esto NO es actividad normal del sistema, se indica una fase de reconocimiento **post-explotacion**.

Respuesta de Q8: nmap

Q9:

¿Cuándo fue el último inicio de sesión del atacante con la IP 219.150.161.20? Formato: MM/DD/AAAA HH:MM:SS AM

19/04/2010 05:56:05 AM

→ Utilice `auth.log` para filtrar las entradas de registro que mencionan la dirección IP `219.150.161.20` e inicios de sesión exitosos.

→ Ejecute `grep "Accepted password" auth.log | grep "219.150.161.20"` para encontrar la hora exacta del último inicio de sesión. Combine esto con el año de los metadatos del archivo para formatear la marca de tiempo correctamente.

La salida muestra que el ultimo inicio de sesión exitoso fue el: **2010/04/19 05:56:05**

```
checho@Windows-localhost:~/Hammered$  grep "Accepted password" auth.log | grep "219.150.161.20"
Apr 19 05:41:44 app-1 sshd[8810]: Accepted password for root from 219.150.161.20 port 51249 ssh2
Apr 19 05:42:27 app-1 sshd[9031]: Accepted password for root from 219.150.161.20 port 40877 ssh2
Apr 19 05:55:20 app-1 sshd[12996]: Accepted password for root from 219.150.161.20 port 55545 ssh2
Apr 19 05:56:05 app-1 sshd[13218]: Accepted password for root from 219.150.161.20 port 36585 ssh2
```

Interpretación: **El 19 de abril a las 05:56:05 el usuario administrador (root) inició sesión con contraseña en el servidor "app-1" desde la IP 219.150.161.20 el 19 de abril.**

¿Como se identifico el AAAA del inicio de sesión?

El archivo de registro auth.log muestra que la ultima modificación se realizo en el año 2010

```
checho@Windows-localhost:~/Hammered$ ls -l
total 13472
drwxr-xr-x 2 root root     4096 Apr 13 19:12 apache2
drwxr-xr-x 2 root root     4096 Apr 13 19:12 apt
-rw-r--r-- 1 root root 10327345 Jul  3  2010 auth.log
-rw-r--r-- 1 root root   115010 Jul  3  2010 daemon.log
-rw-r--r-- 1 root root   228019 Jul  3  2010 debug
-rw-r--r-- 1 root root    35639 May  3  2010 dmesg
-rw-r--r-- 1 root root    36408 Apr 28  2010 dmesg.0
-rw-r--r-- 1 root root    96035 Apr 26  2010 dpkg.log
-rw-r--r-- 1 root root      406 Apr 24  2010 fontconfig.log
drwxr-xr-x 2 root root     4096 Apr 13 19:12 fsck
-rw-r--r-- 1 root root  2479182 Jul  3  2010 kern.log
-rw-r--r-- 1 root root    79695 May  3  2010 messages
-rw-r--r-- 1 root root        0 Apr 25  2010 secure
-rw-r--r-- 1 root root   359696 May  3  2010 udev
-rw-r--r-- 1 root root      213 Mar 18  2010 user.log
```

Respuesta de Q9: **2010-04-19 05:56**

Q10:

La base de datos mostró dos mensajes de advertencia. Indique el más crítico y potencialmente peligroso.

→ Las advertencias de la base de datos suelen registrarse en los registros del sistema, como `daemon.log`. Céntrese en las entradas relacionadas con el servicio de base de datos.

→ Utilice la palabra clave `mysql` para buscar en los registros advertencias relacionadas con la base de datos.

→ Busque advertencias con `grep "mysql" daemon.log | grep "WARNING"`. Una de las advertencias críticas se refiere a cuentas sin contraseña, lo cual supone un riesgo de seguridad significativo.

La salida muestra mensajes de advertencia indicando que **mysql.user contiene 2 cuentas root sin contraseña**

```
checho@Windows-localhost:~/Hammered$ grep "mysql" daemon.log | grep "WARNING"
Mar 18 10:18:42 app-1 /etc/mysql/debian-start[7566]: WARNING: mysql.user contains 2 root accounts without password!
Mar 18 17:01:44 app-1 /etc/mysql/debian-start[14717]: WARNING: mysql.user contains 2 root accounts without password!
Mar 22 13:49:49 app-1 /etc/mysql/debian-start[5599]: WARNING: mysql.user contains 2 root accounts without password!
Mar 22 18:43:41 app-1 /etc/mysql/debian-start[4755]: WARNING: mysql.user contains 2 root accounts without password!
Mar 22 18:45:25 app-1 /etc/mysql/debian-start[4749]: WARNING: mysql.user contains 2 root accounts without password!
Mar 25 11:56:53 app-1 /etc/mysql/debian-start[4848]: WARNING: mysql.user contains 2 root accounts without password!
Apr 14 14:44:34 app-1 /etc/mysql/debian-start[5369]: WARNING: mysql.user contains 2 root accounts without password!
Apr 14 14:44:36 app-1 /etc/mysql/debian-start[5624]: WARNING: mysqlcheck has found corrupt tables
Apr 18 18:04:00 app-1 /etc/mysql/debian-start[4647]: WARNING: mysql.user contains 2 root accounts without password!
Apr 24 20:21:24 app-1 /etc/mysql/debian-start[5427]: WARNING: mysql.user contains 2 root accounts without password!
Apr 28 07:34:26 app-1 /etc/mysql/debian-start[4782]: WARNING: mysql.user contains 2 root accounts without password!
Apr 28 07:34:27 app-1 /etc/mysql/debian-start[5032]: WARNING: mysqlcheck has found corrupt tables
May  2 23:05:54 app-1 /etc/mysql/debian-start[4774]: WARNING: mysql.user contains 2 root accounts without password!
```

Desde un ángulo técnico y de administración de sistemas, tener cuentas de root sin contraseña en MySQL no es solo un descuido, es una vulnerabilidad de severidad crítica que permiten la escalada de Privilegios y control Total.

Los mensajes de advertencia aparecen demasiadas veces, lo indicaría que el problema nunca fue corregido.

Nivel de impacto:

Esto le permite a un atacante:

- **conectarse a MySQL sin credenciales**
- **leer/modificar datos**
- **crear usuarios**
- **incluso escalar a nivel sistema (en ciertos escenarios)**

Respuesta de Q10: mysql.user contains 2 root accounts without password!

Q11:

Se crearon varias cuentas en el sistema de destino. ¿Cuál de ellas se creó el 26 de abril a las 04:43:15?

→ Los registros del sistema, como `auth.log`, rastrean los eventos de creación de cuentas. Se centran en los registros de la fecha y hora especificadas.

→ Busque el comando `useradd` en los registros para encontrar detalles sobre cuándo se crearon las cuentas.

→ Utilice `grep "useradd" auth.log | grep "Apr 26 04:43:15"` para encontrar la cuenta que se creó en ese momento.

Se observa la creación de una nueva cuenta llamada **`wind3str0y`** el **26 de abril a las 04:43:15** mediante el comando **`useradd`**, lo cual es Indicador fuerte de cuenta maliciosa.

```
checho@Windows-localhost:~/Hammered$ grep "useradd" auth.log | grep "Apr 26 04:43:15"
Apr 26 04:43:15 app-1 useradd[20115]: new user: name=wind3str0y, UID=1004, GID=1005, home=/home/wind3str0y, shell=/bin/bash
checho@Windows-localhost:~/Hammered$
```

La cuenta tiene un directorio personal válido **`/home/wind3str0y`** y un intérprete de comandos interactivo **`shell=/bin/bash`**, lo que indica que puede utilizarse para acceder al sistema y ejecutar comandos.

La persistencia se refiere a las técnicas utilizadas por un atacante para mantener el acceso a largo plazo a un sistema comprometido. En este caso, la creación de una nueva cuenta de usuario con una shell interactiva permite al atacante iniciar sesión en cualquier momento sin necesidad de explotar el sistema nuevamente.

Respuesta de Q11: wind3str0y

Q12:

Algunos atacantes usaban un proxy para ejecutar sus análisis. ¿Cuál es el agente de usuario correspondiente que utiliza este proxy?

→ Verifique los registros de acceso del servidor web para detectar cadenas de agente de usuario inusuales, ya que pueden revelar el uso de un proxy o una herramienta de escaneo.

→ El archivo `apache2/www-access.log` contiene información del agente de usuario para las solicitudes entrantes.

→ Extraiga y analice agentes de usuario usando `cat apache2/www-access.log | cut -d ' ' -f 12 | sort | uniq.`

```
checho@Windows-localhost:~/Hammered$ cat apache2/www-access.log | cut -d ' ' -f 12 | sort | uniq
"-"
"Apple-PubSub/65.12.1"
"Mozilla/4.0
"Mozilla/5.0
"WordPress/2.9.2;
"pxyscand/2.1"
```

El análisis de los registros del servidor web reveló varias cadenas User-Agent. Entre ellas, destaca **`pxyscand/2.1`**, ya que está asociada a una herramienta automatizada de escaneo de proxies, lo que indica que el atacante utilizó un método de escaneo basado en proxies.

Respuesta de Q12: pxyscand/2.1